Plan de Implementación: Dual Camera Scan y Fusión Geométrica 3D

1. Visión General del Módulo Dual Camera Scan

El objetivo de este módulo es dotar a la plataforma unificada (`app.py` / `live3d`) de la capacidad de: 1. Previsualizar simultáneamente el campo visual de dos cámaras RealSense en un único visor 3D interactivo WebGL (GPU). 2. Calibrar la relación espacial relativa entre ambas cámaras mediante un asistente visual guiado (patrón ChArUco). 3. Realizar **escaneos estáticos 3D combinados** con un solo clic, fusionando automáticamente las dos nubes de puntos en un único archivo de malla (`.ply` y `.obj`) listo para Blender.

---

2. Procedimiento y Matemáticas de Calibración Extrínseca

Para que los puntos 3D generados por la Cámara 2 (P_2) encajen con los puntos de la Cámara 1 (P_1), es indispensable conocer la matriz de transformación rígida homogénea 4×4 en $SE(3)$:

$$T_{2 \to 1} = \begin{bmatrix} R_{3 \times 3} & t_{3 \times 1} \ 0_{1 \times 3} & 1 \end{bmatrix}$$

donde R es la matriz ortogonal de rotación (3×3) y t es el vector de traslación (3×1) en metros.

```
       [ Cámara 1 (Origen Global) ]
                    |
         T(board -> cam1)
                    v
       [ Tablero ChArUco Común ]
                    ^
         T(board -> cam2)
                    |
       [       Cámara 2           ]

 => T(cam2 -> cam1) = T(board -> cam1) * [ T(board -> cam2) ]^(-1)
```

A. Algoritmo de Calibración con OpenCV ChArUco

Se utiliza un tablero ChArUco porque combina las ventajas de los marcadores ArUco (inmunes a oclusiones y detección inequívoca de orientación) con la precisión subpíxel de las esquinas de un tablero de ajedrez.

```
import cv2
import numpy as np
```

```
def calibrate_relative_pose(img_cam1, img_cam2, intr_1, intr_2):
    """
    Calcula T(cam2 -> cam1) a partir de una captura simultánea del tablero ChArUco.
    """
    # 1. Definir diccionario y tablero (ej. 6x9 cuadros de 30mm, marcadores 22mm)
    aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_5X5_100)
    board = cv2.aruco.CharucoBoard((9, 6), squareLength=0.030, markerLength=0.022, dictionary=
    detector = cv2.aruco.CharucoDetector(board)

    # 2. Detectar esquinas en ambas cámaras
    corners1, ids1, _, _ = detector.detectBoard(img_cam1)
    corners2, ids2, _, _ = detector.detectBoard(img_cam2)

    if corners1 is None or corners2 is None or len(corners1) < 6 or len(corners2) < 6:
        raise ValueError("No se detectaron suficientes esquinas comunes en ambas cámaras.")

    # 3. Estimar pose con solvePnP respecto a cada cámara
    K1 = np.array([[intr_1["fx"], 0, intr_1["ppx"]], [0, intr_1["fy"], intr_1["ppy"]], [0, 0,
    K2 = np.array([[intr_2["fx"], 0, intr_2["ppx"]], [0, intr_2["fy"], intr_2["ppy"]], [0, 0,
    dist1 = np.zeros(5)
    dist2 = np.zeros(5)

    # Pose 1
    obj_pts1, img_pts1 = board.matchImagePoints(corners1, ids1)
    _, rvec1, tvec1 = cv2.solvePnP(obj_pts1, img_pts1, K1, dist1)
    R1, _ = cv2.Rodrigues(rvec1)
    T1 = np.eye(4)
    T1[:3, :3] = R1
    T1[:3, 3] = tvec1.squeeze()

    # Pose 2
    obj_pts2, img_pts2 = board.matchImagePoints(corners2, ids2)
    _, rvec2, tvec2 = cv2.solvePnP(obj_pts2, img_pts2, K2, dist2)
    R2, _ = cv2.Rodrigues(rvec2)
    T2 = np.eye(4)
    T2[:3, :3] = R2
    T2[:3, 3] = tvec2.squeeze()

    # 4. Calcular transformación relativa: T(cam2 -> cam1) = T1 * inv(T2)
    T_2_to_1 = T1 @ np.linalg.inv(T2)
    return T_2_to_1
```

B. Formato de Persistencia (`config/dual_extrinsics.json`)

```
{
 "calibration_date": "2026-10-14 15:30:00",
 "cam_master_serial": "936222071234",
 "cam_slave_serial": "017322075678",
 "matrix_4x4": [
   [0.866, -0.050, 0.497, 0.420],
   [0.020,  0.998, 0.065, 0.015],
   [-0.499, -0.046, 0.865, 0.110],
   [0.000,  0.000, 0.000, 1.000]
```

```
 ],
 "rmse_error_mm": 1.24
}
```

---

3. Pipeline de Fusión y Renderizado Dual

A. Previsualización en Tiempo Real en GPU (Three.js / WebGL)

En lugar de fusionar millones de puntos en la CPU en cada cuadro (lo que destruiría los 30 FPS del stream), la fusión en tiempo real se delega a la GPU mediante el **Vertex Shader de Three.js**:

1. El servidor envía los paquetes de textura de ambas cámaras.
2. La geometría de la Cámara 1 se proyecta con su intrínseca normal.
3. La geometría de la Cámara 2 pasa por la matriz `uExtrinsics` directamente en el Vertex Shader:

```
// Vertex Shader en live3d/web/js/shaders.js
uniform mat4 uExtrinsics;   // Matriz T(cam2 -> cam1)
uniform int uCameraId;      // 1 para Master, 2 para Slave

void main() {
   vec3 localPos = computeDeprojection(uv); // Coordenadas 3D locales
   vec4 worldPos;

   if (uCameraId == 2) {
       worldPos = uExtrinsics * vec4(localPos, 1.0);
   } else {
       worldPos = vec4(localPos, 1.0);
   }

   gl_Position = projectionMatrix * modelViewMatrix * worldPos;
}
```

- **Resultado:** El usuario ve ambas cámaras ensambladas a 30 FPS en tiempo real con latencia casi nula.

B. Fusión Estática de Alta Calidad (Open3D para Archivos en `Saves/`)

Cuando el usuario pulsa **"Captura Dual 3D"**: 1. El hilo de captura toma dos snapshots atómicos y sincronizados: `snap1` y `snap2`. 2. Se construyen dos nubes de puntos RGBD con Open3D: `pcd1` y `pcd2`. 3. Se aplica la matriz de transformación calibrada sobre `pcd2`: `python pcd2.transform(T_2_to_1)` 4. **Refinamiento ICP Fino (Opcional):** Si la zona de solape tiene suficiente textura, se ejecuta un Colored ICP local restringido a traslaciones de ± 5 mm para corregir

micro-desajustes mecánicos de los trípodes. 5. **Fusión y Limpieza:** ```python # Unir nubes pcd_global = pcd1 + pcd2

Unificar densidad en la zona de solapamiento pcd_global = pcd_global.voxel_down_sample(voxel_size=0.002) # 2mm

Eliminar partículas sueltas y ruido en bordes pcd_clean, _ = pcd_global.remove_statistical_outlier(nb_neighbors=25, std_ratio=1.2) `` 6. **Reconstrucción de Malla:** - Se integran las nubes en un volumen **TSDF (Truncated Signed Distance Function)** o se aplica reconstrucción de superficie mediante **Poisson** con recorte por densidad. - Se guarda en Saves//dual_scan_/ : - nube_fusionada.ply (nube limpia con color unificado). - malla_fusionada.obj (malla poligonal sólida lista para Blender). - cam1_color.jpg y cam2_color.jpg . - metadata_dual.json` (matrices extrínsecas e intrínsecas de ambos sensores).

---

4. Endpoints de la API Unificada para Dual Camera Scan

Método	Endpoint	Descripción
`GET`	`/api/dual/devices`	Lista las cámaras detectadas, seriales y roles asignados (Master / Slave).
`POST`	`/api/dual/calibrate`	Activa el asistente de calibración ChArUco y guarda la matriz extrínseca calculada.
`GET`	`/api/dual/extrinsics`	Devuelve la matriz actual de calibración guardada.
`POST`	`/api/dual/capture`	Dispara una captura simultánea, fusiona ambas nubes y genera la malla en `Saves/`.

---

5. Hoja de Ruta de Implementación en Código

```
+-------------------------------------------------------------+
| FASE 1: Driver y Adquisición Dual                           |
| - Crear DualRealSenseSource en live3d/capture.py            |
| - Gestión de seriales, arranque cascada (Slave -> Master)   |
| - Sincronización software / hardware en BaseSource          |
+-----------------------------+-------------------------------+
```

```
                                |
                                v
+-------------------------------------------------------------+
| FASE 2: Calibrador y Herramienta Extrínseca                 |
| - Crear tools/calibrate_charuco.py                          |
| - Guardado y lectura de config/dual_extrinsics.json         |
+------------------------------+------------------------------+
                                |
                                v
+-------------------------------------------------------------+
| FASE 3: Endpoints y Fusión Open3D                           |
| - Añadir rutas /api/dual/* en live3d/server.py              |
| - Función de fusión y denoising dual en Open3D              |
+------------------------------+------------------------------+
                                |
                                v
+-------------------------------------------------------------+
| FASE 4: Frontend y Viewport Dual WebGL                      |
| - Selector "Modo Dual" en live3d/web/index.html             |
| - Carga de matriz uExtrinsics en Shaders WebGL              |
| - Botón "Captura Dual" en el panel lateral                  |
+-------------------------------------------------------------+
```

---

6. Estado Actual de la Implementación: ¡Completado y Verificado!

Las cuatro fases de la hoja de ruta han sido implementadas e integradas satisfactoriamente en el repositorio:

1. **Adquisición Dual Concurrente:**
2. Implementado en `live3d/capture.py` (`DualRealSenseSource`).
3. Gestión inteligente del emisor IR láser: Cámara 1 proyecta patrón para dar textura, Cámara 2 apaga su emisor para evitar interferencia cruzada.
4. Streaming a 30 FPS en ambas cámaras en paralelo.
5. **Calibrador ChArUco Automatizado:**
6. Implementado en `tools/calibrate_dual.py`.
7. Generación y lectura de configuración centralizada en `config/dual_camera_config.json`.
8. Endpoint de calibración directa en un clic desde el navegador: `POST /api/dual/calibrate`.
9. **Fusión Geométrica y Limpieza en Open3D:**
10. Función `capture_dual_scan` en `live3d/capture.py`:
    - Transformación rígida de la nube de Cam 2 mediante $T_{2 \to 1}$.
    - Fusión booleana de nubes de puntos (`pcd1 + pcd2_transformed`).

- Decimación homogénea en zona de solape mediante `voxel_down_sample(voxel_size=0.002)`.
- Supresión de ruido flotante mediante `remove_statistical_outlier(nb_neighbors=25, std_ratio=1.2)`.

11. **Prueba con Hardware Real:** Validada la generación de escaneos duales con **314.843 puntos 3D** almacenados con éxito en la carpeta de sesiones `Saves/`:
    - `nube_fusionada.ply`, `nube_cam1.ply`, `nube_cam2.ply`.
    - `cam1_color.jpg`, `cam2_color.jpg`, mapas de calor de profundidad y `metadata_dual.json`.

12. **Interfaz de Control en el Dashboard Web:**
13. Sección ■ **Modo Dual Activo** visible en `live3d/web/index.html`.
14. Conmutador en vivo para alternar la señal de la Cámara 1 y la Cámara 2.
15. Botón **Captura Dual 360°** con registro instantáneo en el historial de guardados.